ENTREGA TECNICA | DISCIPLINA 02

Pipeline de Dados do MVP CitizenTech

Big Data Architecture & Data Integration

PROJETO
Zela - Zeladoria Urbana por Foto

PAPEL NO MVP
Coletar, tratar e disponibilizar dados para acompanhamento de relatos.

INTEGRANTES DA EQUIPE
Joseh Gabriel Trimboli Agra | RM: 553094
Pedro Henrique de Assuncao Lima | RM: 5527476

Resumo

A solucao implementa um fluxo ETL reproduzivel para transformar relatos de zeladoria em dados analiticos. A carga pode ler a tabela **reports** do MVP ou um CSV de demonstracao, validar e normalizar cada registro, separar rejeicoes e gerar agregacoes por dia, semana e mes.

O pipeline e executavel localmente em Python sem dependencias externas. A orquestracao foi preparada como DAG para Apache Airflow 2.10.5, com Compose e montagem local do banco em modo somente leitura.

O que acompanha esta entrega

• Codigo do pipeline ETL e DAG Airflow

• Configuracao Docker Compose para a aplicacao e o banco local

• Dicionario de dados da origem e das tabelas analiticas

• CSV/JSON gerados e log da execucao local com dados sinteticos

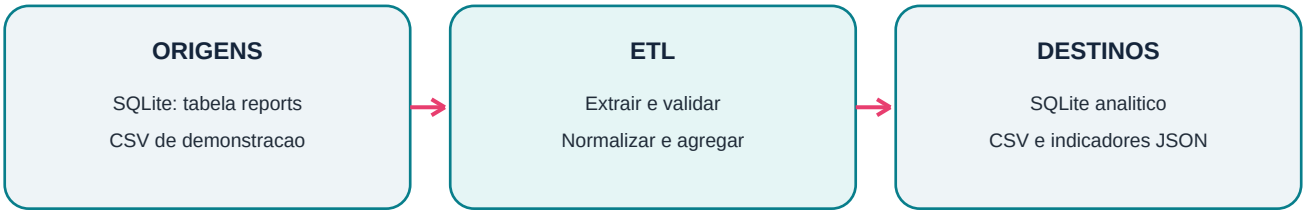
• Instrucoes de reproducao, limites e cuidados de privacidade

Estado da evidencia (atualizado)

Airflow 2.10.5 foi iniciado via Docker Compose. A DAG zela_relatos e as quatro tarefas terminaram SUCCESS; os logs e os arquivos gerados estao incluidos. Falta anexar uma captura da Graph View na interface local.

1. Arquitetura e fluxo

O componente de dados recebe relatos do banco SQLite compartilhado pelo MVP ou de arquivos CSV. A DAG controla as etapas, enquanto o script ETL concentra as regras de validacao e transformacao.



Airflow DAG: preparar fonte -> executar pipeline -> registrar resumo

Etapas do pipeline

Etapa	Responsabilidade
1. Extrair	Ler reports via SQLite ou receber registros CSV; manter identificador report_id.
2. Validar	Conferir campos obrigatorios, datas, categorias, status e coordenadas; registrar motivos de rejeicao.
3. Transformar	Normalizar texto e datas UTC; derivar dia, semana ISO e mes; preservar origem demo/cidadao.
4. Carregar	Recriar tabelas analiticas e gravar registros limpos, rejeitados, agregados e historico de execucao.
5. Orquestrar	DAG Airflow executa as tarefas em ordem e disponibiliza logs por tarefa.

Produtos analiticos

relatos_limpos para consultas por relato; **relatos_rejeitados** para auditoria de qualidade; **agregado_periodo** para contagens e indicadores por categoria, status e origem; **execucoes** para rastreabilidade da carga.

Desenho simplificado: origem -> extracao e validacao -> transformacao -> destino analitico. Os arquivos gerados podem ser inspecionados sem abrir o Airflow.

2. Contrato de dados e privacidade

O contrato comum permite que o aplicativo e as disciplinas usem a mesma chave. A tabela de origem e **reports**; o identificador **id** e exposto como **report_id**.

Camada	Campos e regras principais
Origem reports	id inteiro; created_at UTC; location aproximada; latitude/longitude opcionais; description; category; status; source; campos opcionais da visao computacional e ordem de servico.
relatos_limpos	id, created_at normalizado, dia/semana/mes, local, coordenadas validas, categoria/status/origem canonicos, metadados de visao e indicador de ordem de servico.
relatos_rejeitados	Campos necessarios para corrigir a qualidade, sem imagem ou segredo; coluna motivos lista as validacoes que falharam.
agregado_periodo	periodo_tipo, periodo, category, status, source, total e indicadores de revisao, concordancia e encaminhamento.
execucoes	executado_em, run_id, fonte e indicadores_json para rastrear cada carga.

Regras importantes

• O dado de demonstracao usa source=demo, separado dos relatos cidadaos.

• A DAG nao extrai o nome do arquivo de imagem nem confirm_token.

• A camada analitica nao copia descricao livre; localizacao detalhada e reduzida ao necessario para a analise.

• Coordenadas incompletas ou fora dos limites geograficos sao descartadas na transformacao.

• Os indicadores mostrados nesta entrega foram gerados com dados sinteticos e nao descrevem ocorrencias reais.

O dicionario completo, com tipos, dominios e descricoes de cada coluna, esta no repositorio em **zela_bigdata/DICIONARIO_DE_DADOS.md**.

3. Execucao, evidencia e reproducao

A execucao local do script ETL usou o arquivo reports_demo.csv. O conjunto de demonstracao tem 40 linhas e e identificado como sintetico.

40	40	0	40
registros lidos	validos	rejeitados	marcados demo

Indicadores calculados na demonstracao: taxa de confirmacao pelo cidadao 0,95; concordancia da sugestao de visao 0,974; taxa encaminhada 0,475. Como os 40 registros sao sinteticos, esses valores servem somente para validar o processamento.

Reproduzir a execucao local

Na pasta disciplinas/02-bigdata, rode:

```
python zela_bigdata\dags\zela_pipeline.py --fonte zela_bigdata\exemplos\reports_demo.csv --saida
zela_bigdata\evidencias\execucao_local
```

Executar a DAG no Airflow

Resultado real da execucao

DAG zela_relatos executada via Docker Compose com o CSV sintetico. Run ID **manual__entrega_bigdata_20261110**: SUCCESS. Tarefas **extrair, validar, agregar e carregar**: todas SUCCESS.

Resultado: 40 relatos lidos, 40 validos, 0 rejeitados, 40 demo; os logs CSV/JSON estao incluidos. **Testes automatizados: 8 passaram.**

Repositorio publico da equipe: github.com/Zoviskgabriel/GenAI-Zeladoria-Urbana-por-Foto. O pacote entregue inclui codigo, configuracao, dicionario de dados, evidencias e este relatorio.

Captura pendente: Graph View do Airflow. A execucao e comprovada pelos logs; ainda falta salvar a imagem da interface local em http://localhost:8080 apos login.